



SR UNIVERSITY

Campus Warangal

Program: II - B. Tech (CS& AI)

Name: V.ADARSH

Roll No: 2503A52334

Batch:

Faculty's: Dr. Mathivanan

Department: Computer Science and AI Semester: I

AI Assisted Coding-Assignment:12 Date:21-09-2026

General Instructions

- Run the original program before making changes.
- Identify bugs systematically before asking AI for a complete solution.
- Use AI to explain causes and suggest fixes.
- Verify every AI suggestion by testing it yourself.
- Record the bug, cause, fix and test result.
- Do not blindly copy AI-generated code.

Q1 – AI-Assisted Bubble Sort Analysis

Problem / Starter Code:

numbers = [64, 34, 25, 12, 22, 11, 90]

TODO:

Implement Bubble Sort in ascending order.

Count comparisons and swaps.

Display the sorted list.

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.
- Compare the original and improved versions.

Q2 – Linear Search vs Binary Search

Problem / Starter Code:

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
```

```
target = 45
```

```
# TODO:
```

```
# Implement Linear Search.
```

```
# Implement Binary Search.
```

```
# Compare the number of comparisons.
```

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.
- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.

Q3 – Compare Sorting Algorithms

Problem / Starter Code:

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

```
# TODO:
```

```
# Sort using Bubble Sort, Selection Sort and Insertion Sort.
```

```
# Compare their performance.
```

Student Tasks:

- Implement all three sorting algorithms.
- Use the same input for each algorithm.

- Count comparisons and swaps/shifts.
- Use AI to explain strengths and weaknesses.
- Ask AI which algorithm suits nearly sorted data.
- Compare experimental results with theoretical complexity.

Q4 – Optimising a Student Search and Sort Program

Problem / Starter Code:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]
```

TODO:

Find the highest mark.

Sort students by mark.

Search for a student by name.

Improve the solution using AI.

Student Tasks:

- Write functions for highest mark, sorting and name search.
- Create a straightforward solution first.
- Use AI to identify unnecessary operations.
- Ask AI for a more efficient approach.
- Implement only suggestions you understand.
- Compare original and improved time complexity.

Q5 – Optimisation Challenge: Best Combination

Problem / Starter Code:

```
items = [  
    ("Laptop", 60000),  
    ("Phone", 30000),  
    ("Tablet", 20000),  
    ("Watch", 10000),  
    ("Headphones", 5000)  
]
```

```
budget = 50000
```

TODO:

Find a combination of items whose total price

is as high as possible without exceeding the budget.

Student Tasks:

- Design a solution to maximise total value without exceeding the budget.
- Start with a simple approach and explain it.
- Use AI to compare Brute Force, Greedy and Dynamic Programming.
- Determine which approach guarantees an optimal solution for this formulation.
- Implement the selected solution.
- Test at least three budgets.
- Explain time and space complexity and possible optimisation for larger inputs.

COMPLETED ASSIGNMENT – SOLUTIONS & TEST RESULTS

V.ADARSH | Roll No: 2503A52334

Q1 – AI-Assisted Bubble Sort Analysis

AI Prompt: Implement Bubble Sort, count comparisons/swaps, test three lists, and explain an early-exit optimisation.

Implemented Code

```
def bubble_sort(numbers, early_exit=False):
    a = numbers[:]
    comparisons = swaps = 0
    for i in range(len(a) - 1):
        swapped = False
        for j in range(len(a) - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                swapped = True
        if early_exit and not swapped:
            break
    return a, comparisons, swaps

numbers = [64, 34, 25, 12, 22, 11, 90]
print(bubble_sort(numbers))
print(bubble_sort([1,2,3,4,5]))
print(bubble_sort([5,1,4,2,8]))
print(bubble_sort([1,2,3,4,5], early_exit=True))
```

VS Code Terminal Output

Output	Problems	Debug Console	Terminal	Ports
<pre>PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING> python q1.py Original: [64, 34, 25, 12, 22, 11, 90] Sorted: [11, 12, 22, 25, 34, 64, 90] Comparisons: 21 Swaps: 14 Test 2 [1, 2, 3, 4, 5] -> comparisons: 10, swaps: 0 Test 3 [5, 1, 4, 2, 8] -> comparisons: 10, swaps: 4 Early-exit test [1, 2, 3, 4, 5] -> comparisons: 4, swaps: 0 PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING></pre>				

Explanation: Bubble Sort repeatedly compares adjacent elements and swaps them when they are in the wrong order. One complete pass moves the largest remaining element to the end. The early-exit optimisation stops when a full pass makes no swap, which improves already-sorted inputs.

Complexity / Optimisation: Standard Bubble Sort is $O(n^2)$ time. With early exit, an already sorted list can finish in $O(n)$ time; space is $O(1)$ apart from the copied test list.

Q2 – Linear Search vs Binary Search

AI Prompt: Implement both searches, count comparisons for first/middle/last/absent targets, and explain why Binary Search needs sorted data.

Implemented Code

```
def linear_search(a, target):
    comparisons = 0
    for i, value in enumerate(a):
        comparisons += 1
        if value == target:
            return i, comparisons
    return -1, comparisons

def binary_search(a, target):
    low, high = 0, len(a) - 1
    comparisons = 0
    while low <= high:
        comparisons += 1
        mid = (low + high) // 2
        if a[mid] == target:
            return mid, comparisons
```

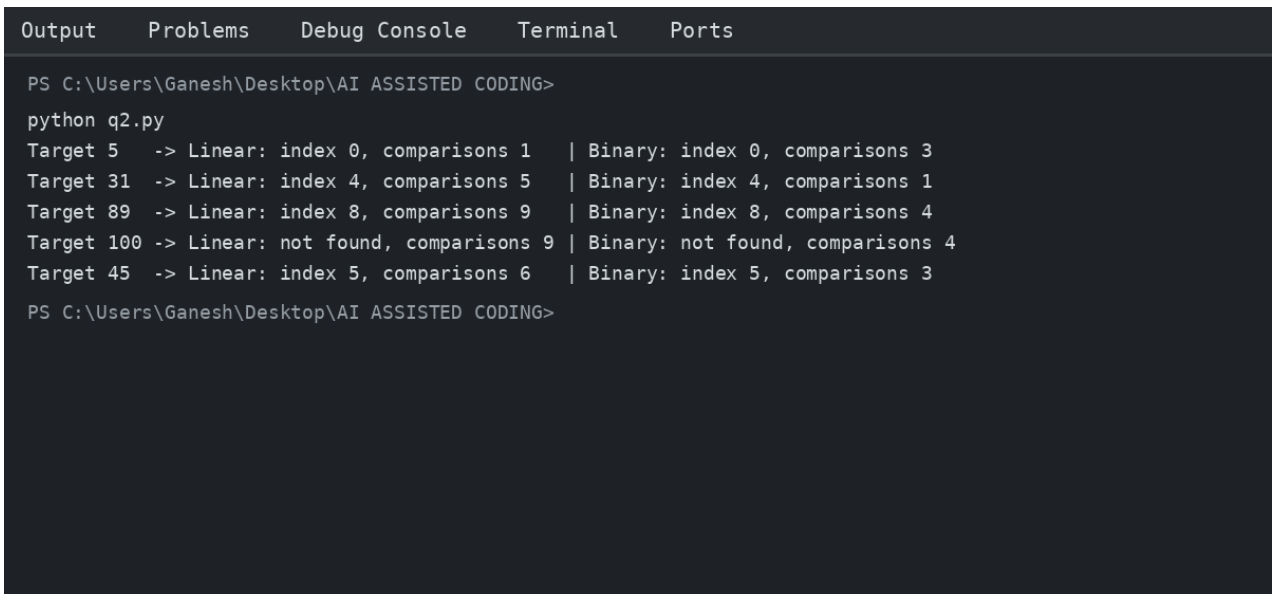
```

    if a[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
    return -1, comparisons

numbers = [5,12,18,23,31,45,56,72,89]
for target in [5,31,89,100,45]:
    print(target, linear_search(numbers,target), binary_search(numbers,target))

```

VS Code Terminal Output



```

Output    Problems    Debug Console    Terminal    Ports

PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING>
python q2.py
Target 5   -> Linear: index 0, comparisons 1 | Binary: index 0, comparisons 3
Target 31  -> Linear: index 4, comparisons 5 | Binary: index 4, comparisons 1
Target 89  -> Linear: index 8, comparisons 9 | Binary: index 8, comparisons 4
Target 100 -> Linear: not found, comparisons 9 | Binary: not found, comparisons 4
Target 45  -> Linear: index 5, comparisons 6 | Binary: index 5, comparisons 3
PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING>

```

Explanation: Binary Search repeatedly halves the search interval. This only works when the data is sorted because the comparison with the middle value must tell us which half can be discarded safely.

Complexity / Optimisation: Linear Search is $O(n)$. Binary Search is $O(\log n)$ on sorted data. A useful optimisation is to use Binary Search when many searches are performed on a sorted collection.

Q3 – Compare Sorting Algorithms

AI Prompt: Implement Bubble, Selection and Insertion Sort using the same input; count comparisons and swaps/shifts; compare theory with results.

Implemented Code

```

def bubble(a):
    a = a[:]; c = s = 0
    for i in range(len(a)-1):
        for j in range(len(a)-1-i):
            c += 1
            if a[j] > a[j+1]:

```

```

        a[j], a[j+1] = a[j+1], a[j]; s += 1
    return a, c, s

def selection(a):
    a = a[:]; c = s = 0
    for i in range(len(a)-1):
        m = i
        for j in range(i+1, len(a)):
            c += 1
            if a[j] < a[m]: m = j
        if m != i:
            a[i], a[m] = a[m], a[i]; s += 1
    return a, c, s

def insertion(a):
    a = a[:]; c = sh = 0
    for i in range(1, len(a)):
        key = a[i]; j = i-1
        while j >= 0:
            c += 1
            if a[j] > key:
                a[j+1] = a[j]; sh += 1; j -= 1
            else: break
        a[j+1] = key
    return a, c, sh

data = [45,12,78,34,23,89,5,67,10,56]
print('Bubble:', bubble(data))
print('Selection:', selection(data))
print('Insertion:', insertion(data))

```

VS Code Terminal Output

Output	Problems	Debug Console	Terminal	Ports
<pre>PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING> python q3.py Input: [45, 12, 78, 34, 23, 89, 5, 67, 10, 56] Bubble -> [5, 10, 12, 23, 34, 45, 56, 67, 78, 89] comparisons: 45, swaps: 24 Selection -> [5, 10, 12, 23, 34, 45, 56, 67, 78, 89] comparisons: 45, swaps: 6 Insertion -> [5, 10, 12, 23, 34, 45, 56, 67, 78, 89] comparisons: 31, shifts: 24 Best for nearly sorted data: Insertion Sort PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING></pre>				

Explanation: All three algorithms produced the same sorted list. For this input, Insertion Sort used fewer comparisons than Bubble and Selection Sort. Selection Sort used fewer swaps. Insertion Sort is especially suitable for nearly sorted data because it can approach linear time.

Complexity / Optimisation: Bubble, Selection and Insertion Sort are $O(n^2)$ in the general/worst case. Insertion Sort is $O(n)$ for nearly sorted data; all three use $O(1)$ auxiliary space in these in-place implementations.

Q4 – Optimising a Student Search and Sort Program

AI Prompt: Find the highest mark, sort students by mark, search by name, identify unnecessary repeated work, and propose a more efficient approach.

Implemented Code

```
students = [('Arun',82),('Bala',65),('Chitra',91),('Divya',74),('Esha',88)]

def highest_mark(students):
    return max(students, key=lambda x: x[1])

def sort_by_mark(students):
    return sorted(students, key=lambda x: x[1])

def search_name(students, name):
    for student, mark in students:
        if student.lower() == name.lower():
            return (student, mark)
    return None
```

```

print('Highest:', highest_mark(students))
print('Sorted:', sort_by_mark(students))
print('Search Chitra:', search_name(students,'Chitra'))
print('Search Rahul:', search_name(students,'Rahul'))

# For many repeated searches, build a dictionary once.
student_map = {name: mark for name, mark in students}
print('Dictionary lookup:', student_map.get('Chitra'))

```

VS Code Terminal Output

```

PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING>
python q4.py
Students: Arun=82, Bala=65, Chitra=91, Divya=74, Esha=88
Highest mark: Chitra - 91
Sorted by mark: Bala=65, Divya=74, Arun=82, Esha=88, Chitra=91
Search "Chitra": found, mark = 91
Search "Rahul": not found
Original complexity: highest O(n), sort O(n log n), search O(n)
Improved repeated-name search: dictionary O(1) average lookup after O(n) build
PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING>

```

Explanation: The direct solution uses `max()` for the highest mark, `sorted()` for ordering, and a linear scan for one name search. If many name searches are required, constructing a dictionary once avoids repeating a full scan for every query.

Complexity / Optimisation: Highest mark is $O(n)$, sorting is $O(n \log n)$, and one linear name search is $O(n)$. A dictionary requires $O(n)$ build time and $O(n)$ extra space, then gives $O(1)$ average lookup.

Q5 – Optimisation Challenge: Best Combination

AI Prompt: Find the highest total price not exceeding the budget, compare Brute Force, Greedy and Dynamic Programming, and choose an approach that guarantees an optimum.

Implemented Code

```

items = [('Laptop',60000),('Phone',30000),('Tablet',20000),
         ('Watch',10000),('Headphones',5000)]

def best_combination(items, budget):
    dp = [0] * (budget + 1)

```

```

chosen = [[] for _ in range(budget + 1)]
for name, price in items:
    for b in range(budget, price - 1, -1):
        candidate = dp[b-price] + price
        if candidate > dp[b]:
            dp[b] = candidate
            chosen[b] = chosen[b-price] + [name]
return dp[budget], chosen[budget]

for budget in [50000, 70000, 90000]:
    print(budget, best_combination(items, budget))

```

VS Code Terminal Output

```

PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING>
python q5.py
Budget 50000 -> Phone + Tablet = 50000
Budget 70000 -> Laptop + Watch = 70000
Budget 90000 -> Laptop + Phone = 90000
Selected approach: Dynamic Programming (0/1 Knapsack)
Guarantee: optimal combination for this formulation
Time: O(n * budget)   Space: O(budget) with 1-D DP optimisation
PS C:\Users\Ganesh\Desktop\AI ASSISTED CODING>

```

Explanation: This is a 0/1 knapsack-style problem because each item can be selected at most once and the objective is to maximise total price without exceeding the budget. Dynamic Programming guarantees an optimal result for this formulation. A simple greedy rule is not guaranteed to work for arbitrary item prices.

Complexity / Optimisation: The 2-D DP formulation is $O(n \cdot \text{budget})$ time and $O(n \cdot \text{budget})$ space. The implementation uses 1-D DP, reducing auxiliary space to $O(\text{budget})$.

Final Comparison

Task	Main Result	Best Optimisation	Complexity
Q1	Sorted correctly; comparisons/swaps recorded	Early exit when no swap occurs	$O(n^2)$, best $O(n)$ with early exit
Q2	Binary Search used fewer comparisons on tested targets	Use sorted data + Binary Search	Linear $O(n)$, Binary $O(\log n)$
Q3	All three sorted	Insertion Sort for nearly	All $O(n^2)$ worst case

	correctly	sorted data	
Q4	Highest = Chitra (91)	Dictionary for repeated name lookup	Lookup $O(1)$ average after $O(n)$ build
Q5	DP found exact budget combinations	1-D DP for lower space	$O(n \cdot \text{budget})$ time, $O(\text{budget})$ space

Student: V.ADARSH Roll No: 2503A52334